Group 3: Iteration 2 Report

ECE 452 Software Engineering

Authors: Rishita Dasgupta, Jasleen Chhatwal, Julia Alvarado, Mehal Bhattacharya

Danielle Yosifovich, Vaanika Singla

Role Assignment

Requirements Team: Danielle Yosifovich and Mehal Bhattacharya.

Development Team: Julia Alvarado and Jasleen Chhatwal.

Testing Team: Rishita Dasgupta and Vaanika Singla.

Requirements Team

User Stories + Features

1. **Forgot Password:** "As a user, I want to be able to reset my password if I forget it, so that I can regain access to my account and AI conversations."

- **SMART Goal**
    - **Specific:** The system will provide a mechanism for users to request a password reset link via their registered email address.
    - **Measurable:** The system must validate the email format and send a password reset email within <5 seconds of a valid request.
    - **Achievable:** The feature will be implemented using a secure token generation process and an email routing service to deliver the reset link.

    - **Relevant:** It ensures the user will not permanently lose access to their account, preserving the ability to view chat history.
    - **Time-boxed:** The forgot password feature and UI flow will be implemented during Iteration 2.
- **Acceptance Criteria**
    - A user can click a "Forgot Password" link located on the Login page.
    - The system prompts the user to enter their registered email address.
    - If the email exists in the database, the system sends a temporary, secure password reset link to that email.
    - For security purposes, the system displays a generic confirmation message ("If an account exists, a reset link has been sent") regardless of whether the email was found.
    - Clicking the reset link navigates the user to a secure "Reset Password" page to input and confirm a new password.
- **Story points:** 2 points

2. **Star Conversations:** “As a user, I want to star or pin specific conversations (chats) so that I can quickly find and continue my most important previous conversations.”

- **SMART Goal**
    - **Specific:** The system shall allow users to toggle a "star" status on any saved conversation listed in their history sidebar.
    - **Measurable:** Starred conversations must visually update immediately upon clicking and dynamically sort to the top of the history list in <2 seconds.

    - **Achievable:** The feature will be implemented by adding a boolean **isStarred** field to the database schema for saved conversations.
    - **Relevant:** This addresses the core customer need to keep, view, and continue previous conversation history efficiently.
    - **Time-boxed:** The pinning feature will be implemented during Iteration 2.
- **Acceptance Criteria**
    - A visible "yellow star" icon appears next to each conversation title in the history view.
    - Clicking the icon toggles the state between starred and unstarred.
    - Starred conversations are visually distinct (a filled-in icon rather than an outline).
    - Starred conversations are automatically pinned to a dedicated section at the top of the history list.
    - The pinned status is saved to the database and persists across different user login sessions.
- **Story points:** 1 point

3. **Save Conversation History**: “As a user, I want to keep my conversation history so I can view and search it , as well as continue my previous conversations.”

- **SMART Goal**
    - **Specific:** The system will automatically save all chat messages between the user and the LLM to the database, organizing them into discrete sessions.

- **Measurable:** The system must save messages continuously without explicit user action and successfully fetch/render the history list in <3 seconds upon page load.
- **Achievable:** The feature will be implemented by sending the chat data payload to a REST API endpoint that stores it securely in the backend database linked to the user's ID.
- **Relevant:** This directly fulfills the core customer requirement to view, retain, and eventually search past interactions.
- **Time-boxed:** The save history backend functionality and frontend rendering will be implemented during Iteration 2.

- **Acceptance Criteria**
  - Every message sent by the user and generated by the AI is automatically saved to the current active session.
  - A logged-in user can view a chronological list of their past conversation sessions on the screen.
  - Clicking on a previous conversation successfully loads that specific chat history into the main chat window.
  - The user can send new messages to a loaded historical chat to continue the conversation.
  - The history remains persistent and accessible across multiple login/logout cycles.
- **Story points:** 3 points

Lo-Fi UI Features

1. **Forgot Password Page**

- **Purpose:** Allow users to initiate a password reset process to regain account access securely.
- **Contains:**
    - Page title (e.g., "Reset Your Password")
    - Brief instructional text
    - Email input field
    - "Send Reset Link" submit button
    - "Back to Login" navigation link
- **Navigation Flow:** Login Page → Forgot Password Page → Email Confirmation UI

2. **Star Conversations (Sidebar Element)**

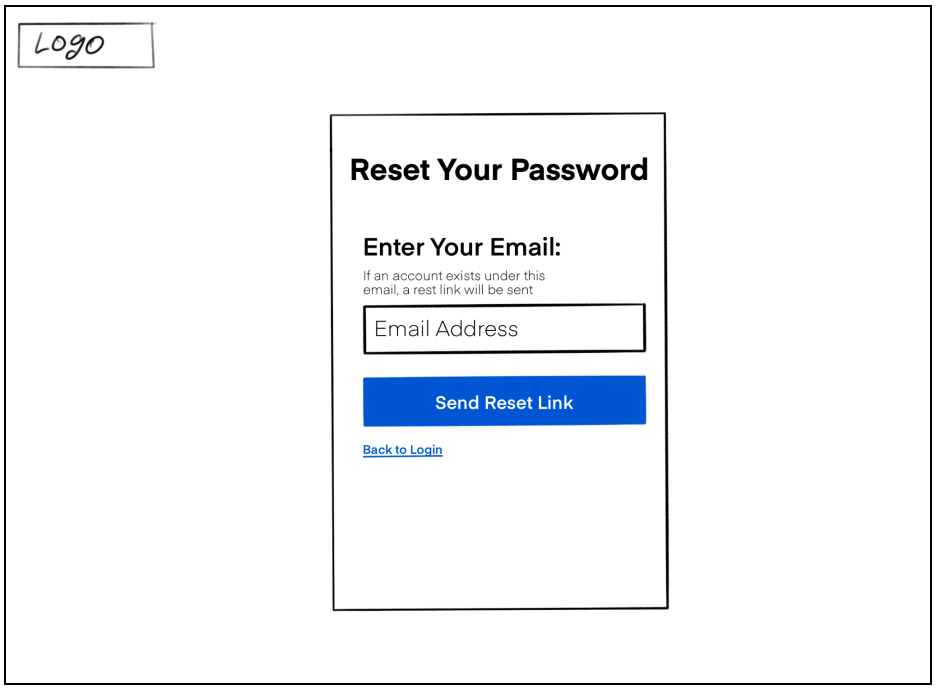
- **Purpose:** Allow users to highlight, organize, and easily access important past AI interactions.
- **Contains:**
    - Interactive Star toggle icon next to each chat log entry.
    - A dedicated "Starred" header category at the top of the history sidebar to separate them from standard chronological history.
- **Navigation Flow:** Chat Interface → Click Star Icon → UI immediately updates to move the conversation to the top "starred" section.

3. **Conversation History Sidebar**

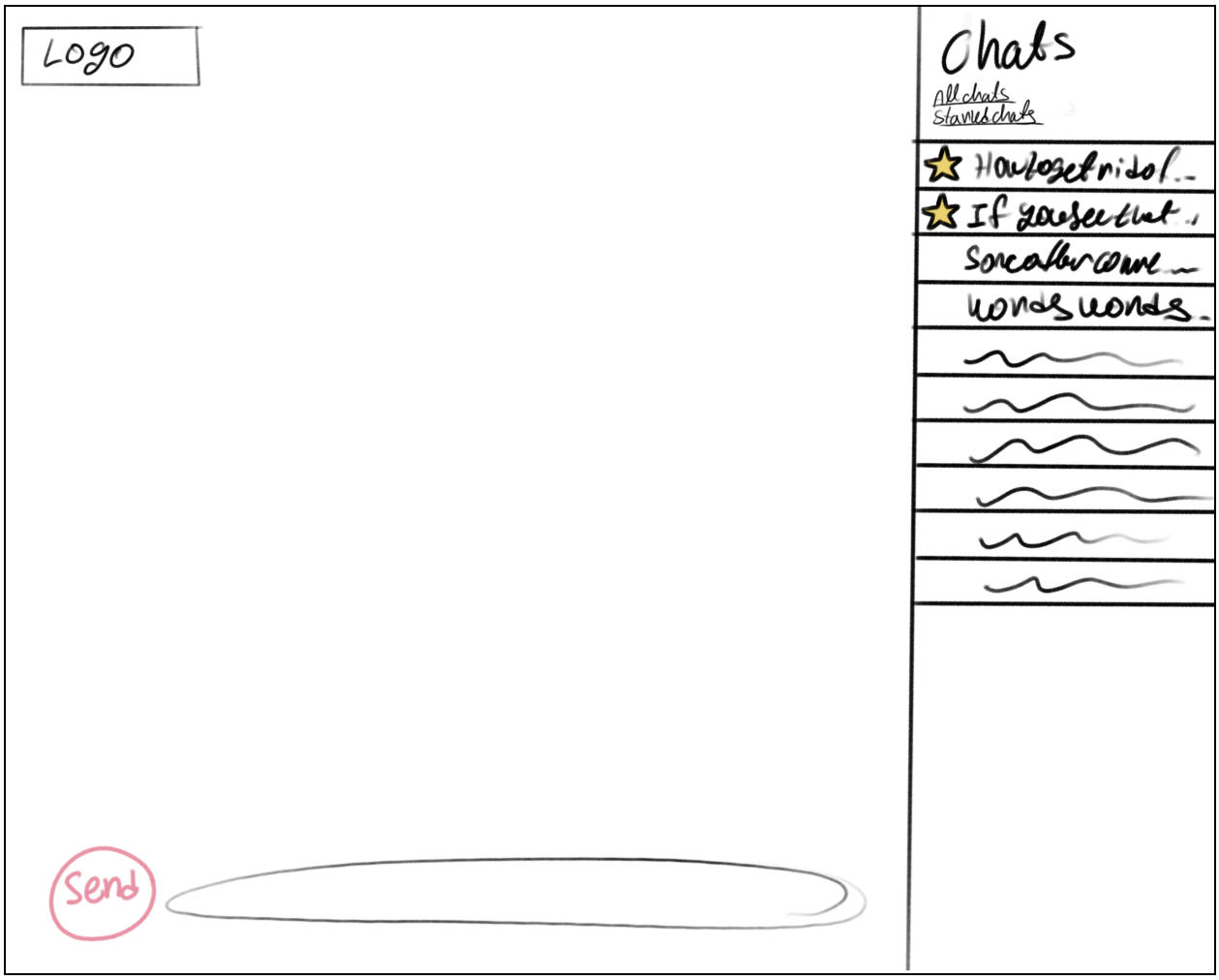
- **Purpose:** Provide a navigational menu to display a list of saved conversations, allowing the user to view or resume them.

- **Contains:**
    - A "New Chat" button to start a fresh session.
    - A scrollable list of past conversation titles (e.g., named by the first prompt).
    - (Integration with previous feature): The Star/Pin icon next to each entry.
- **Navigation Flow:** History Sidebar → Click a previous conversation title → Main chat interface clears current text and populates with the saved messages from the database.

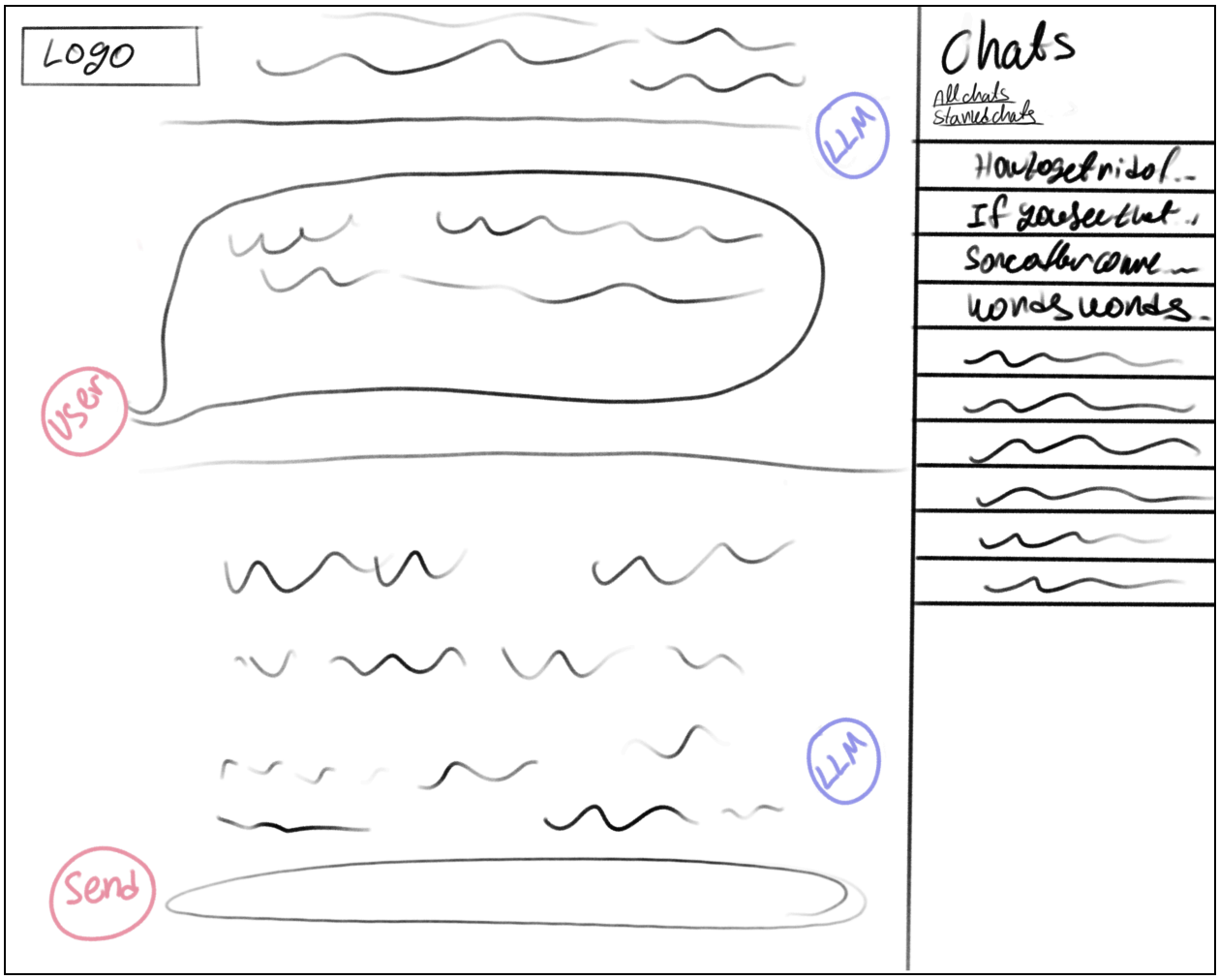
1. “Forgot the password” page. Was drawn on iPad using Procreate



2. “Star Conversation” page. Was drawn on iPad using Procreate



3. “Chat History” page. Was drawn on iPad using Procreate.



The github repo: https://github.com/rdasgupta-3/soft-eng-group3

Development Team

1. Number of Features Implemented
    a. Forgot Password Feature: A complete password reset flow was implemented. Clicking the Forgot Password button on the main page will take the user to a Forgot Password page, where they can request a reset link by inputting their user email. Clicking on the reset password link will navigate the user to the Reset Password page where they can set a new password. Sending the link via email was not implemented due to restrictions on what we can keep in the repository, so instead of using a separate email service, the reset link is shown directly in the UI.
    b. Conversation History: A chat history was implemented so users could access or continue their past chats even after logging out of the app or closing the tab. The Database design was modified so that conversations are now saved in **conversations.json**, allowing users to access their conversation history even after the server is restarted.
    c. Favorite/Search/ Delete Conversations: Users can now favorite any of their chats by using the star button in the conversation history sidebar. The favorited chats then go into their own “Starred” section. Users can also delete chats that they no longer need and search for specific chats using the search bar.
    d. Ollama Detection and Warm-Up: A new system was implemented to detect whether Ollama is running locally. If Ollama is available at server startup, the backend performs an automatic warm-up. If Ollama starts later (after server

started), the frontend detects it via **/api/ollama-status** and triggers **/api/warmup**. If Ollama is offline, the chat UI disables the chat input and displays a warning banner telling the user to download and start Ollama. This feature was added not only to inform users if Ollama is running, but also to reduce the time for a first reply from Ollama by triggering a warm-up.

e. General Chat updates and timeout: The chat UI was updated with a typewriter animation for AI responses and a " ___ is thinking…" indicator while waiting for the model. A timeout was also implemented so that Ollama would abort their response if they took more than 30 seconds to load. If Ollama timesout, a user will receive a fallback message in the chat instead of the AI response.

2. Unit Tests Implemented Before Coding (TDD Approach):

We wrote Jasmine unit tests for the loginPage, signupPage, forgotPasswordPage, resetPasswordPage, and chatPage. We followed TDD by writing the unit tests before implementing features, and then ensured our logic met the test conditions. All our tests load one page in the test environment, call the page functions, and check if the behavior is correct. Each file is organized as a test suite and each behavior is checked as a test case.

For the loginPage unit test, we tested that if the user enters an invalid email the page shows an error message and does not call the backend. In order to do this we used jasmine.createSpy('fetch'), which acted like a fake fetch function. When we fill in the email and password fields, and call attemptLogin(), we verify that the error message appears and that fetch was not called. Additionally, we verified that a valid

email/password does call the backend by calling /api/login.

For the signupPage unit test, we first tested that the page loads, and then we had it replace fetch with a Jasmine spy, enter an invalid email and password, call createAccount, and confirm that an error is visible and that the signup API was not called. This tests the input validation on the creating account page.

For the forgotPasswordPage unit test, we verified that, if the email format is invalid, the page displays an error and does not make a 'forgot password' API call. This test was also necessary to implement as part of our Iteration 2 'forgot password' story, which requires the system to prompt for an email and the unit tests ensure that the email is valid before making the request to the backend API.

For the resetPasswordPage unit test, we tested to check that, if there was no token in the URL, submitting the reset form shows an error saying the reset token is missing. We also tested that if the new password and confirmation do not match, the page shows a mismatch error. This test was also necessary to implement as part of our Iteration 2 forgot password story, and this was a good test because it checks the reset page's input and URL validation logic.

For the chatPage unit test, we tested chat history and conversation management logic. We tested to ensure that, when the page loads, it automatically creates an empty conversation in localStorage and that the messages area starts empty. We also tested that creating a new conversation does not create another one if the current chat is still empty. In addition, we tested to make sure that, after a user message is added, a second conversation can be made. Other tests were, if a conversation can be pinned and the

pinned state is saved, and that a conversation can be deleted and that the stored list updates accordingly. This unit test was also necessary to implement as part of our Iteration 2 stories for starring conversations and saving conversations history.

3. Iteration Process: Unit Tests to Acceptance Tests
    a. The unit tests were organized into Jasmine suites by page so that each feature could be checked independently. We used 5 tests, loginPage.spec.js, signupPage.spec.js, forgotPasswordPage.spec.js, resetPasswordPage.spec.js, and chatPage.spec.js. For the login, signup, and forgot password pages, we tested that invalid email input shows an error before any backend call is made. For reset password, we tested for missing tokens and mismatched passwords. For chat, we tested conversation history behavior like creating chats, preventing duplicate empty chats, and starring and deleting chats. These tests helped us verify functionality before moving to full acceptance testing.
    b. After unit tests passed, we started focusing on passing acceptance tests and resolving issues brought up by the testing team. Since many of the issues pertained to fixing database logic or creating more logic for the backend and frontend to run Ollama properly, tests were done manually in the server until all issues were resolved. After our changes, we then went back and re-ran the acceptance tests until everything was resolved.
4. REST API routing Table and Database Design

The images pasted below is the README of our Github Repository, which has our updated Routing Table, Database Design as well as some other information about our implemented features.

System Architecture

REST API Routing Table

Below is the current routing table for our Express.js backend, separating our frontend page routes from our backend API endpoints.

Frontend Page Routes

Method	Endpoint	Purpose
GET	`/`	Renders the Landing/Login page
GET	`/signup`	Renders the Account Creation page
GET	`/choose-player`	Renders the Persona Selection screen
GET	`/chat`	Renders the main LLM Chat interface
GET	`/forgot-password`	Renders the Forgot Password Page
GET	`/reset-password`	Renders the Reset Password page

Authentication API Routes

Method	Endpoint	Purpose
POST	`/api/signup`	Accepts `{email, password}` to register a new user
POST	`/api/login`	Validates `{email, password}` to authenticate users
POST	`/api/logout`	Logs out the current user and clears the session
POST	`/api/forgot-password`	Generates a password reset token and logs a reset link
POST	`/api/reset-password`	Validates a reset token and updates the user's password

Chat + System API Routes

Method	Endpoint	Purpose
GET	`/api/ollama-status`	Checks if Ollama is running
GET	`/api/warmup`	Warms up the Ollama model to reduce first-reply latency
GET	`/api/conversations`	Returns all conversations for the logged-in user
POST	`/api/conversations`	Creates a new conversation
PATCH	`/api/conversations/:conversationId`	Updates conversation metadata for favoriting
DELETE	`/api/conversations/:conversationId`	Deletes a conversation
POST	`/api/conversations/:conversationId/messages`	Adds user input or AI responses into chat
POST	`/api/conversations/:conversationId/ai-reply`	Generates an AI reply using Ollama

Database Design

For the current development iteration, we are utilizing a **File-Based JSON Storage** so that the user data is persistent across server restarts. This allows login, logout, password reset, and conversations to persist even if the server restarts. These files are loaded and saved using Node's fs module into our `/data` directory.

File	Purpose
users.json	Stores registered users email and password
sessions.json	Stores active login sesssions
resetTokens.json	Stores password reset tokens with expiration timestamps
conversations.json	Stores all users conversations

Entity: User

- `email` (String) - acts as the unique identifier/username.
- `password` (String) - currently stored as plaintext for QA testing (Note: Password hashing via bcrypt will be implemented in future iterations).

Entity: Conversation

- `id` - unique conversation ID
- `email` - owner of the conversation
- `messages[]` - array of `{type, text, at}`
- `pinned` - boolean
- `updatedAt` - timestamp

Local Ollama Chat Integration

This project can generate AI chat replies from a local Ollama instance (no API key required).

Setup

1. Install Ollama from the official site: https://ollama.com
2. Install a model (default used by this project):
   - `llama3.2:latest`
3. Pull the model
   - `ollama pull llama3.2:latest`
4. Start Ollama (default local server):
   - `ollama serve`

Default Environment Variables

You can override the default environment variables using an .env in the project root. If no `.env` is provided, the backend defaults to:

- `OLLAMA_BASE_URL = http://127.0.0.1:11434`
- `OLLAMA_MODEL = llama3.2:latest`

The frontend checks `/api/ollama-status` to determine whether Ollama is running. If Ollama is offline or unavailable, the chat UI displays a warning banner and disables message input. `/api/conversations/:id/ai-reply` also throws an error: `503 { "error": "ollama-failed" }`.

Ollama Warm-Up Behavior

To reduce slow first responses, the backend performs **two warm-ups**. The first is an automatic warm-up on server start, performed in `server.js` after a short delay. The second is a Frontend-triggered warmup, which calls `/api/warmup` to perform a warmup when ollama is detected by `/api/ollama-status`.

Ollama Timeout

If Ollama takes more than 30 seconds to respond, `/api/conversations/:id/ai-reply` will throw an error: `503 { "error": "ollama-failed" }`. The front end will then display a fallback message, which is still saved into conversation history:

> "Sorry, I'm having trouble responding right now. Please try again."

Testing Team

soft-eng-group3 / features /

Add file ···

vaanikas06 Made changes to tests 730fc26 · 5 hours ago History

This branch is 3 commits ahead of and 5 commits behind `main`. Contribute

Name	Last commit message	Last commit date
..		
step_definitions	Made changes to tests	5 hours ago
support	Made changes to tests	5 hours ago
auth-login.feature	standardize test credentials across iteration 2 suite	3 days ago
auth-logout.feature	standardize test credentials across iteration 2 suite	3 days ago
auth-protected-routes.feature	standardize test credentials across iteration 2 suite	3 days ago
chat-conversation-switching.feature	standardize test credentials across iteration 2 suite	3 days ago
chat-history-empty-states.feature	standardize test credentials across iteration 2 suite	3 days ago
chat-history-management.feature	standardize test credentials across iteration 2 suite	3 days ago
chat-history-persistence.feature	standardize test credentials across iteration 2 suite	3 days ago
chat-history-search.feature	standardize test credentials across iteration 2 suite	3 days ago
chat-history-ui.feature	standardize test credentials across iteration 2 suite	3 days ago
chat-send-message.feature	standardize test credentials across iteration 2 suite	3 days ago
chat-session-selection.feature	standardize test credentials across iteration 2 suite	3 days ago
chat.feature	reverted login.feature back to original, also took out chat response ...	3 weeks ago
forgot-password.feature	standardize test credentials across iteration 2 suite	3 days ago
login.feature	reverted login.feature back to original, also took out chat response ...	3 weeks ago
reset-password.feature	standardize test credentials across iteration 2 suite	3 days ago
routing-navigation.feature	standardize test credentials across iteration 2 suite	3 days ago
select.feature	standardize test credentials across iteration 2 suite	3 days ago

soft-eng-group3 / features / step_definitions /

Add file ···

vaanikas06 Made changes to tests 730fc26 · 5 hours ago History

This branch is 3 commits ahead of and 5 commits behind `main`. Contribute

Name	Last commit message	Last commit date
..		
chatSteps.js	fix failing tests	yesterday
conversationHistorySteps.js	Add step definitions for the acceptance tests	4 days ago
forgotPasswordSteps.js	Add step definitions for the acceptance tests	4 days ago
iteration2Steps.js	Made changes to tests	5 hours ago
loginSteps.js	fix failing tests	yesterday
selectSteps.js	fix failing tests	yesterday
timeout.js	Test: Fixed QA test selectors, seeded in-memory DB, and updated asser...	3 weeks ago

soft-eng-group3 / features / support /

Add file ···

vaanikas06 Made changes to tests 730fc26 · 5 hours ago History

This branch is 3 commits ahead of and 5 commits behind `main`. Contribute

Name	Last commit message	Last commit date
..		
authTestUtils.js	fix failing tests	yesterday
chatCleanup.js	fix failing tests	yesterday
serverLifecycle.js	fix failing tests	yesterday
timeout.js	added timeout file	3 weeks ago
world.js	Made changes to tests	5 hours ago

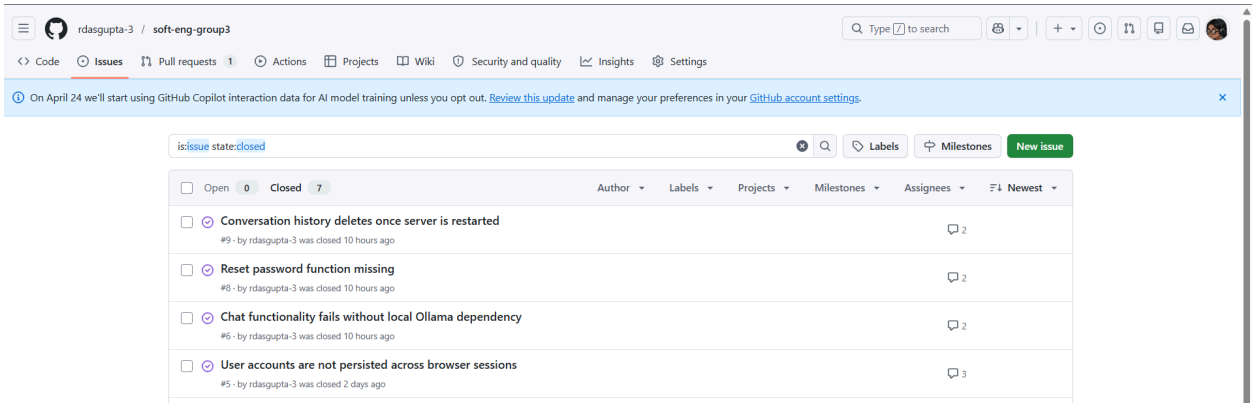
The acceptance tests generated by Cucumber.js to validate the features implemented by the development team are based on 4 separate test suites. There are 17 feature files that encompass 40 scenarios, which are intended to cover user-facing flows of the application. There were 4 core pillars we looked to organize our scenarios into:

- Password recovery suite
- Routing and navigation suite
- Chat interaction suite
- Conversation history suite

The scenarios were derived straight from the main use cases of the chat application. The conversation history use case is structured based on the fact that the chat page shows the conversation list, search field, and new conversation button. Additionally, previous conversations are still visible via a sidebar to the left even after creating a new chat. Conversations also persist even after refreshing the local host session. In our tests, search filters previous conversations by text, and empty search results display an empty-state message. Another way we determined acceptance tests was examining conversation management. We created tests that checked if users could pin conversations, delete conversations, and switch between conversations to view the correct thread contents. Our acceptance tests are strong because they check both natural “happy paths” / default scenarios as well as edge/error cases. For example, we test all sorts of reset-password scenarios: invalid token, short password, and empty password. We text empty chat message submission and searches with no matches in addition to arbitrary test messages. These are important because we are testing that the development team not only have working features, but that their features fail correctly as well.

We organized individual tests into test suites primarily by functional area, using at least 1 .feature file per subsystem. The password recovery suite focuses on password recovery workflow, including form validation, valid email submission, reset token handling, and password constraints. The routing and navigation suite focuses on page loading, route access, and transitioning into the chat experience after selecting a personality. The chat interaction suite verifies that sending prompts and checking that submitted text appears in the chat interface. The conversation history suite is the largest and reflects Iteration 2's main focus of preserving, managing, searching, and switching conversation history. This organization helps with readability, but in future iterations we could definitely organize the test suites more clearly and remove overlapping .feature files.

For interacting via Github issues, we raised issues based on each major testing area and referenced both our acceptance tests generated by Cucumber.js as well as manual testing. In each issue, we described the expected behavior and the reproduced failures. We raised 4 issues, which were promptly resolved by the development team.



Additionally, as per the requirements of Iteration 2, we also conducted a pull request (PR) review that the development team posed. This pull request involved merging the

development team's branch with main. As a result, we created a standardized testing branch to methodically test as described previously. This way, we would not bloat the development team's features with various .feature files and step definition .js tests. After running our tests on the development team's features and verifying the result, we were able to accept the PR review into main. Our testing branch remains separate for readability.

Automated browser tool demo can be found on

https://github.com/rdasgupta-3/soft-eng-group3